Lập trình mạng · Chương 3

HTTP/2: binary framing và multiplexing

TS. Đặng Ngọc Hùng

Trang web cần hàng chục tài nguyên, HTTP/1.1 chỉ một request mỗi kết nối

1. Một trang web cần **hàng chục tài nguyên** (ảnh, script, stylesheet) từ cùng một server
2. HTTP/1.1: trên một kết nối, request sau **chờ response trước**; trình duyệt mở khoảng **6 kết nối** mỗi server
3. Thêm kết nối: thêm bắt tay, thêm handshake TLS, mà vẫn chỉ 6 request cùng lúc

> **Câu hỏi của phần:** làm sao chuyển nhiều request song song trên chỉ một kết nối TCP?

HTTP/1.1 cần 4 lượt cho 20 request, HTTP/2 chạy cả 20 stream trong một lượt

HTTP/1.1: 6 kết nối, mỗi kết nối một request tại một thời điểm



4 lượt ≈ 4 × 200 ms

request sau chờ response trước trên cùng kết nối

HTTP/2: 1 kết nối, 20 stream song song

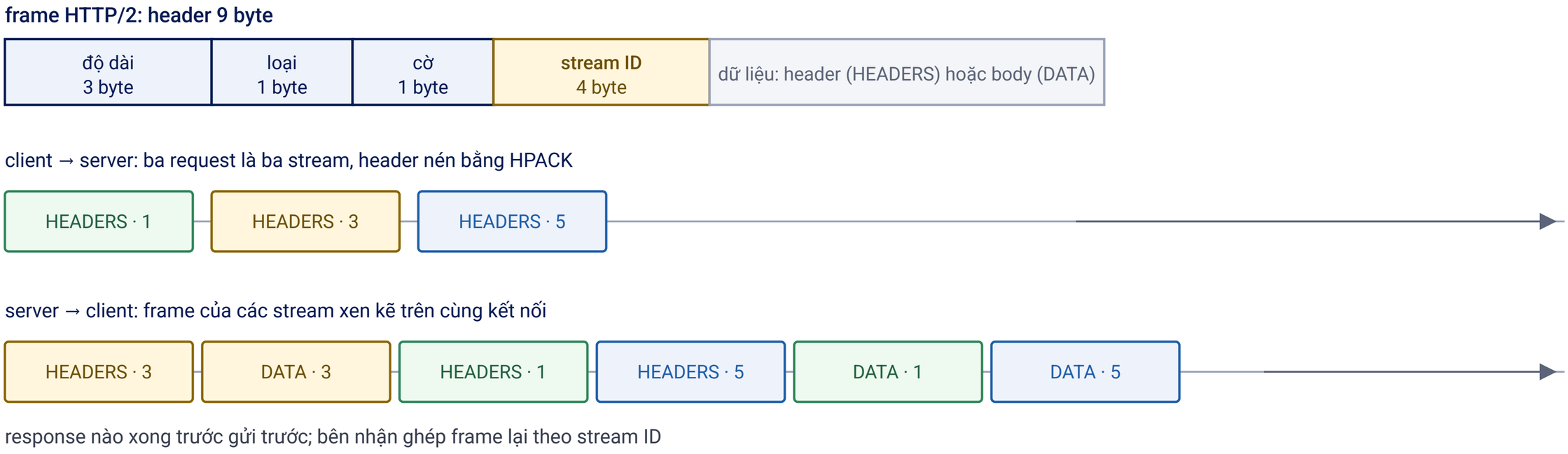


1 lượt ≈ 200 ms

r1 … r20 là 20 stream trên cùng kết nối

Multiplexing: nhiều **HTTP/2 stream** chạy song song trên một kết nối TCP; mỗi stream là một cặp request–response.

Binary framing: frame mang stream ID, frame các stream đi xen kẽ



Binary framing: frame nhị phân mang stream ID; HEADERS chứa header nén bằng **HPACK**, DATA chứa body; frame các stream đi xen kẽ.

HttpClient chọn HTTP/2: 20 request, một kết nối, khoảng 260 mili giây

Java

```
HttpClient client = HttpClient.newBuilder()
        .version(http11 ? HttpClient.Version.HTTP_1_1 : HttpClient.Version.HTTP_2)
        .build();
get(client, "/warmup");                    // request đầu: với h2c, kết nối được nâng cấp lên HTTP/2 ở đây
try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 1; i <= 20; i++) {
        String path = "/r" + i;
        pool.submit(() -> {
```

Kết quả chạy

```
HTTP/2, at most 20 at a time: 20 requests in 261 ms, response version [HTTP_2]
server: 20 requests on 1 connection(s), protocol [HTTP/2.0]
```

Ép HTTP/1.1 với 6 kết nối: cùng 20 request, chậm hơn 3 lần

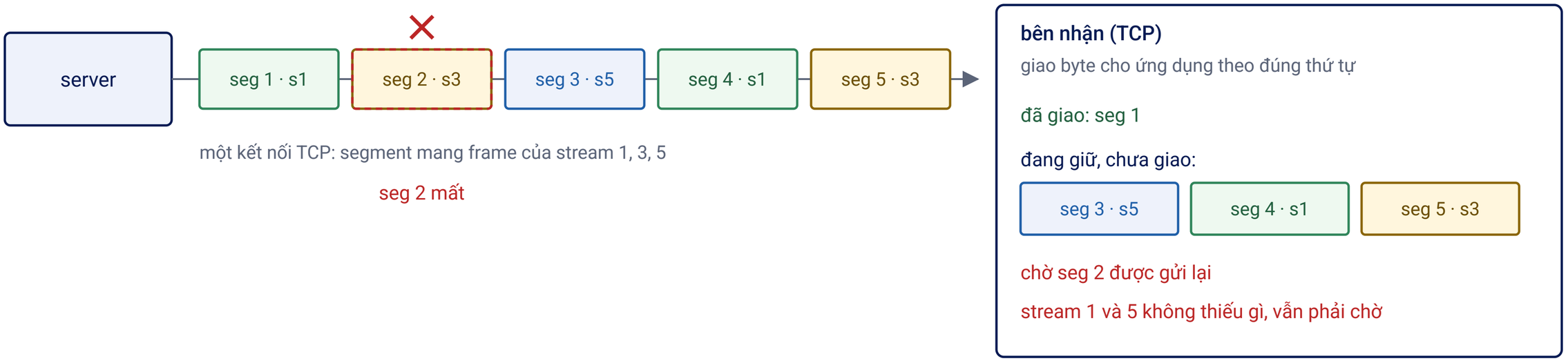
```java
int limit = args.length > 1 ? Integer.parseInt(args[1]) : http11 ? 6 : 20;
Semaphore slots = new Semaphore(limit);    // HTTP/1.1: mặc định 6 request cùng lúc, như trình duyệt
            slots.acquire();
            try {
                versions.add(get(client, path).version());
            } finally {
                slots.release();
```

Kết quả chạy

```
HTTP/1.1, at most 6 at a time: 20 requests in 863 ms, response version [HTTP_1_1]
server: 20 requests on 6 connection(s), protocol [HTTP/1.1]
```

- Bỏ giới hạn (`ParallelFetch 1.1 20`): 267 ms, nhưng **20 kết nối**

Head-of-line blocking còn lại ở tầng TCP: mất một segment, mọi stream chờ



mất một segment của stream 3 → mọi stream trên kết nối phải chờ: head-of-line blocking ở tầng TCP

Head-of-line blocking còn lại ở tầng TCP: mất một segment làm mọi HTTP/2 stream trên kết nối phải chờ.

Câu hỏi 1

Client gửi 20 request tới cùng một server bằng HTTP/2. Client mở bao nhiêu kết nối TCP?

A. Một kết nối; 20 request là 20 HTTP/2 stream trên kết nối đó.

B. 20 kết nối, mỗi request một kết nối.

C. 6 kết nối, như trình duyệt với HTTP/1.1.

D. Số kết nối bằng số nhân CPU của client.

Câu hỏi 1

Client gửi 20 request tới cùng một server bằng HTTP/2. Client mở bao nhiêu kết nối TCP?

A. Một kết nối; 20 request là 20 HTTP/2 stream trên kết nối đó.

B. 20 kết nối, mỗi request một kết nối.

C. 6 kết nối, như trình duyệt với HTTP/1.1.

D. Số kết nối bằng số nhân CPU của client.

HTTP/2 ghép nhiều stream trên một kết nối (multiplexing); mỗi request là một stream có stream ID riêng. Giới hạn 6 kết nối là cách trình duyệt bù cho HTTP/1.1.

Câu hỏi 2

Trên một kết nối HTTP/2, một segment TCP mang dữ liệu của stream 3 bị mất. Các stream khác bị ảnh hưởng thế nào?

A. Không ảnh hưởng, vì mỗi stream độc lập.

B. Chỉ stream 3 chờ segment được gửi lại.

C. Kết nối HTTP/2 bị đóng và mở lại.

D. Mọi stream trên kết nối phải chờ, vì TCP chỉ giao byte cho ứng dụng theo đúng thứ tự.

Câu hỏi 2

Trên một kết nối HTTP/2, một segment TCP mang dữ liệu của stream 3 bị mất. Các stream khác bị ảnh hưởng thế nào?

A. Không ảnh hưởng, vì mỗi stream độc lập.

B. Chỉ stream 3 chờ segment được gửi lại.

C. Kết nối HTTP/2 bị đóng và mở lại.

D. Mọi stream trên kết nối phải chờ, vì TCP chỉ giao byte cho ứng dụng theo đúng thứ tự.

Các stream độc lập ở tầng HTTP/2 nhưng đi chung một byte stream TCP. TCP giữ các segment đến sau segment bị mất cho tới khi segment đó được gửi lại, nên mọi stream phải chờ: head-of-line blocking ở tầng TCP.

Tổng kết

1. **HTTP/2**: một kết nối, nhiều **HTTP/2 stream** song song (**multiplexing**)
2. **Binary framing**: frame nhị phân mang stream ID; header nén bằng **HPACK**
3. **Head-of-line blocking** còn lại ở tầng TCP: mất một segment, mọi stream chờ

Từ khóa cần nhớ HTTP/2 · binary framing · HTTP/2 stream · multiplexing · HPACK · head-of-line blocking

Mở rộng

Chủ đề nâng cao

- Server push và lý do trình duyệt bỏ hỗ trợ
- Độ ưu tiên của stream

Bài tập mở rộng

Đo thời gian tải 50 tài nguyên nhỏ bằng HttpClient với HTTP/1.1 và HTTP/2; trình bày kết quả và giải thích.

Tiếp theo

HTTP/3: QUIC và giao thức trên UDP